WHITEPAPER

Governing AI Agents at the MCP Boundary

Least-privilege enforcement and tamper-evident audit for the Model Context Protocol — deployment patterns for organizations of every size

For security, infrastructure, and AI platform leaders

July 2026 · Version 1.0

eunolabs

github.com/eunolabs/eunox

Executive summary

AI agents are moving into production, and the **Model Context Protocol (MCP)** has become the standard way they reach real systems: databases, payment APIs, CRMs, file stores, messaging. Identity for that traffic is a solved problem — your IdP and OAuth stack already answer who is calling and whether they may access a server at all. What remains unsolved is **behavior**: what a well-authenticated agent may actually **do** through a tool — call by call, argument by argument, across a session — and how you **prove afterward** what it did.

eunox answers that second question. It is a policy-enforcement proxy for MCP: one static binary that sits between any MCP host and any MCP server, checks every enforced call against a declarative capability manifest before forwarding, denies by default anything the manifest does not grant, and writes every decision — allow and deny — to an HMAC-signed, tamper-evident audit log. It governs servers you did not write and cannot modify, and it composes with, rather than replaces, your existing identity infrastructure.

This paper is written for the people who have to make the call: the CISO asked to sign off on agent access to production data, the platform lead asked to run it, and the AI lead asked to move fast without creating the next incident. It describes what an eunox deployment looks like at four organizational sizes — and one pattern for any company that ships an MCP server of its own.

> **The adoption path is the same at every size — and the first step is reversible.** Observe what your agents actually call (audit-only, zero blast radius), draft a least-privilege policy from the observed traffic, enforce it fail-closed in the request path, then manage many proxies as one governed fleet. You adopt visibility first; enforcement earns its way in with evidence.

Key takeaways

- **OAuth is necessary, not sufficient.** Scopes admit a caller to a server; they cannot bound arguments, budgets, or call sequences — and most local MCP traffic carries no token at all.
- **The characteristic agent failure is a well-authenticated agent doing the wrong thing.** Prompt injection does not steal credentials; it spends yours, inside the granted scope.
- **Session memory is the differentiator.** The canonical exfiltration is two individually-valid calls: read credentials, then write externally. Only a stateful monitor in the request path can refuse the second call because of the first.
- **Evidence is half the product.** Every decision lands in a signed, hash-chained audit record that can be verified as untampered — a record your auditors, examiners, and incident reviewers can trust independent of any server's application logs.
- **Start free, start reversible.** The proxy, every enforcement feature, and the audit chain are Apache-2.0 open source and run entirely on your infrastructure.

1. The problem: identity is solved, behavior is not

Wiring an agent to MCP servers multiplies its reach: one assistant can hold live connections to a warehouse, a CRM, a payments API, and your source control at once. Every one of those connections is typically authenticated correctly — and that is precisely why the residual risk is so easy to underestimate. Four gaps separate “the agent authenticated” from “the agent is governed.”

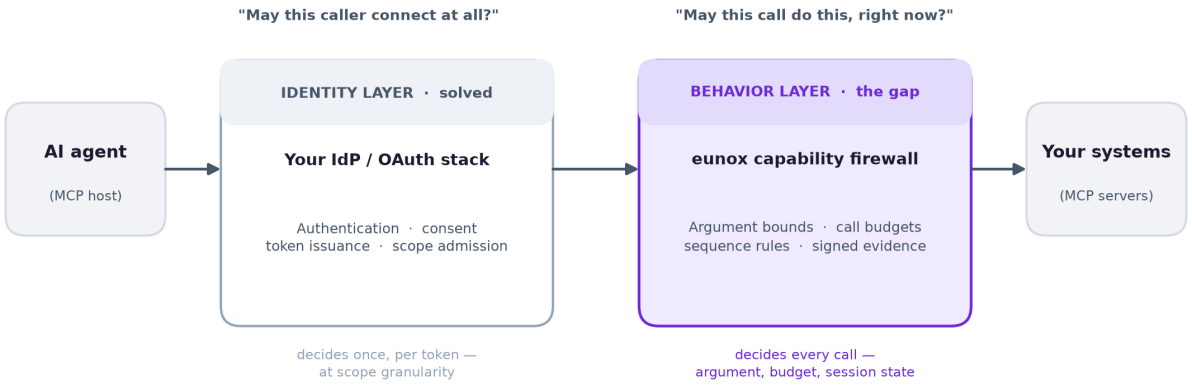

Figure 1 — Identity decides who may connect; behavior must be decided on every call. eunox composes with the identity layer rather than replacing it.

1.1 Scopes stop at the door

An OAuth scope can grant `payments:write`; it cannot express “SELECT-only SQL,” “only case and task records, never accounts,” or “at most five orders per hour.” Scope decisions are made once, at token issuance, at whatever granularity the server author chose. An LLM-driven caller fails differently than a human or a script: a prompt-injected refund or an over-eager bulk update happens **entirely inside the granted scope**, and no identity control will object.

1.2 No per-request engine remembers the session

Agent risk is often a **sequence**, not a call. Each step is individually in scope, in budget, and authorized; the combination is the attack. Tokens, scopes, WAFs, and stateless policy engines evaluate one request at a time and structurally cannot see it (Section 2.2 shows how eunox does).

1.3 Most MCP traffic has no OAuth in the path at all

A large share of real-world MCP usage is a desktop host spawning a local server as a subprocess over stdio. There is no token, no resource server, and no consent screen anywhere in that path. An argument that “MCP OAuth is enough” is an argument about remote HTTP servers only; the larger local surface is untouched by it.

1.4 Enforcement and evidence live in code you don't control

Even where OAuth applies, a scope is enforced by whatever the third-party server author implemented — invisible to you, inconsistent across servers, and logged however they chose. When the incident review asks

what exactly the agent did last Tuesday, a folder of heterogeneous application logs — each editable by whoever operates the server — is not an answer a security organization can stand behind.

> **The right mental model:** your IdP is the identity layer — who gets in the door. eunox is a **reference monitor and flight recorder** — what a caller may do once inside, and signed proof of what it did. Organizations have never treated IAM as a substitute for egress control or audit; agents make the behavioral layer urgent, not optional.

2. eunox: a reference monitor for MCP

eunox is a single static Go binary with no required infrastructure. It speaks MCP on both sides, so it drops into the path of any host and any server — local subprocess or remote HTTP — without changing either. Policy is a YAML **capability manifest**: a deny-by-default allowlist in which a tool that is not granted simply does not exist for the agent.

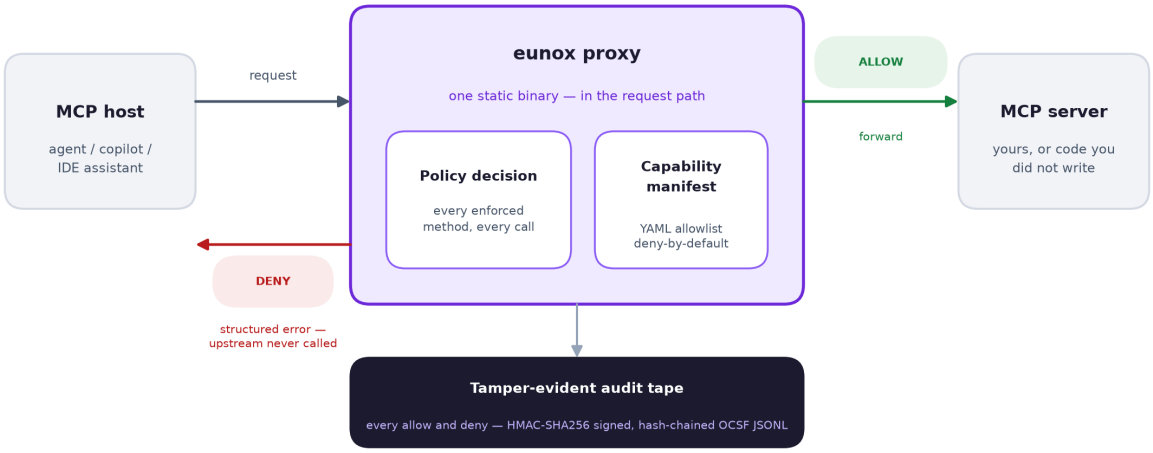

Figure 2 — The enforcement path. A denied call is answered with a structured error; the upstream is never invoked. Every decision is recorded either way.

Property	What it means for a decision-maker
Deny by default	Anything not explicitly granted is refused. A new tool added by a server upgrade is denied until a human allows it.
Fail closed	Missing policy, malformed token, unknown method, unavailable counter backend — every ambiguity resolves to a structured deny, never to “forward and hope.”
Typed constraints	Argument allowlists, SQL-verb restrictions, per-session call budgets, sequence rules, time windows, source-IP ranges, recipient-domain checks. A policy typo is a load error, not a silently weaker policy.
Response redaction	PII/PCI fields are masked in responses before the model ever sees them (`redactFields`).
Tamper-evident audit	Every allow and deny is an HMAC-SHA256-signed, hash-chained OCSF record. `eunox audit-verify` proves the log was not edited or truncated mid-chain.
IdP composition	Validates your IdP's JWTs (signature, expiry, issuer, audience); token claims can only narrow the manifest, never expand it.
Zero telemetry	The binary makes no background network calls. What runs in your perimeter answers only to you.

2.1 The attack per-request engines cannot see

One capability deserves its own picture, because it is the clearest illustration of why the behavior layer must live in the request path and keep state. Consider an agent that reads a credential store (legitimate) and later posts to an external endpoint (legitimate). Both tools are in policy. The pair moves secrets out of your perimeter.

Figure 3 — A sequence rule denies the external write for the rest of any session in which credentials were read. Both tools remain available to sessions that use only one of them.

Per-session call budgets (`maxCalls`) work the same way: they bound the blast radius of a misbehaving loop even when every individual call is valid. Both controls depend on state carried across requests — which is why neither a token nor a stateless gateway rule can express them.

3. The adoption path

Every deployment in Section 4 walks the same four stages. The operational split that matters is not “dev versus prod” but **observe** (reversible, audit-only) versus **enforce** (fail-closed, in the request path). Crossing that line is a decision you make with a week of real evidence in hand, not on faith.

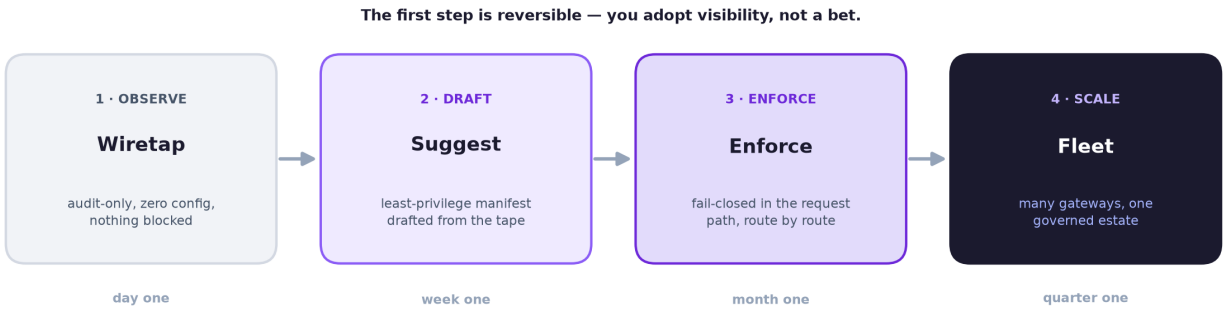

Figure 4 — Wiretap, suggest, enforce, fleet. Each stage pays for the next: the tape you collect in stage 1 becomes the draft policy of stage 2.

- **Observe.** `eunox proxy --audit -- <server command>` wraps any existing server in one line. Nothing is blocked; every call lands on the signed tape. Reading a day of it is usually the moment agent governance stops being abstract.

- **Draft.** `eunox suggest` proposes a least-privilege manifest from the observed traffic — including argument-value constraints built from what the agent actually passed. A human reviews it like any other change.
- **Enforce.** The reviewed manifest goes live route by route. Denials return structured errors the agent can understand; the upstream is never touched on a denied call.
- **Fleet.** Gateways multiply across teams; policy distribution, cross-host audit aggregation, and an organization-wide kill switch become the operating concern (Section 4.3).

4. Deployment profiles by organization size

The profiles below differ in scale and ceremony, not in kind. Find the row that looks like you in the summary table (Section 5), then read that profile — each one states the deployment shape, a representative policy, and the trigger that graduates you to the next profile.

4.1 Small engineering team (up to ~50 engineers)

Situation. A handful of engineers wiring agents — a coding assistant, an internal copilot, an automation — to MCP servers they mostly did not write. No security team, no procurement, no appetite for new infrastructure.

Deployment. One binary, two shapes, an afternoon of work. First, the audit-only wiretap under whatever the host already spawns. Second, a guard in front of any high-value remote server (payments, CRM): default posture read-only, every destructive or money-moving tool **denied by absence**, writes opted in one at a time. The database policy below is the canonical example — the agent can query; it cannot mutate:

```
# warehouse.yaml — the agent queries; it never mutates
capabilities:
  - target: tool:query
    actions: [call]
    conditions:
      - type: allowedOperations
        argument: sql
        operations: [SELECT]      # no INSERT / UPDATE / DELETE / DROP
      - type: maxCalls
        count: 200
        windowSeconds: 3600
  # every write tool: absent from this file -> denied
```

What you get. The agent cannot refund a customer, drop a table, or mass-delete records — regardless of what it was prompt-injected into attempting — and you hold a signed record of everything it did. **What you deliberately skip:** Redis, JWT, HA, fleet tooling. None of it applies at this size, and the binary does not ask you to think about it.

Graduate when a second team appears, or an agent touches a system with compliance obligations.

4.2 Mid-size company (~50–500 engineers)

Situation. Multiple teams run agents against real internal systems; there is an IdP (Okta, Entra, Auth0) and SOC 2 pressure; someone in platform or security engineering now owns the question “what are all these agents doing to production data?” — and per-developer, per-machine wrappers cannot answer it.

Deployment. The **gateway** replaces per-server wrappers: one eunox process fronts every server a team's agents talk to, each on its own route with its own reviewed, versioned policy, all writing one shared tape.

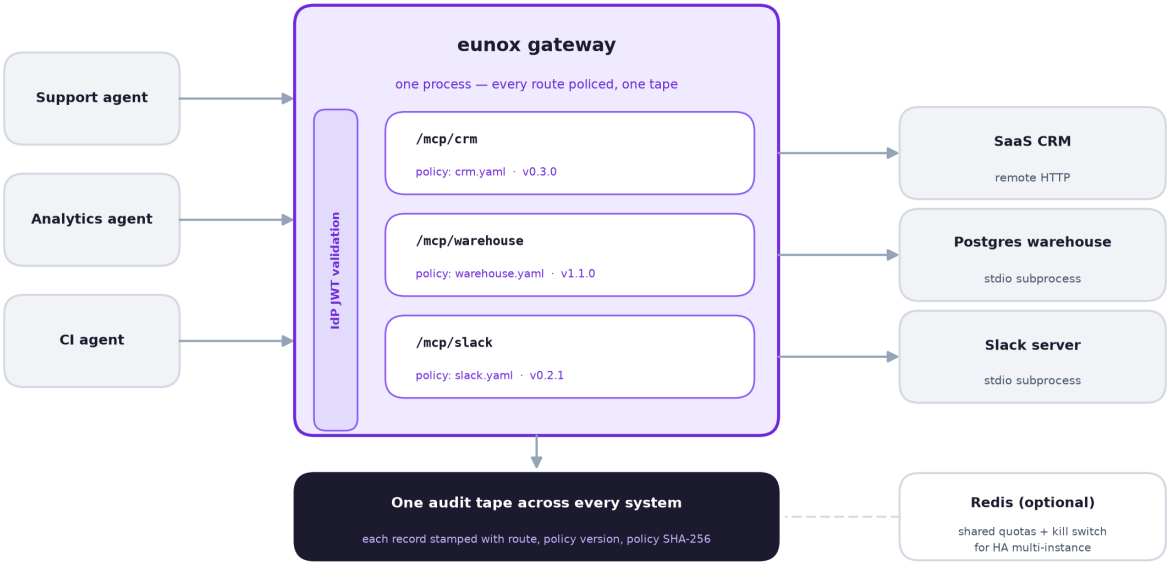

Figure 5 — Gateway mode. One process, one audit tape; each route carries its own independently versioned policy. Redis is added only when you run multiple instances.

```
# gateway.yaml — one proxy, the team's tool surface, one audit tape
schemaVersion: "0.1"
listen:   { bind: 127.0.0.1, port: 3000, authToken: ${EUNOX_GATEWAY_TOKEN} }
audit:    { log: /var/log/eunox/audit.jsonl }
defaults: { enforcement: audit }   # observe first; flip per route
upstreams:
  - name: crm                      # -> POST /mcp/crm
    transport: http
    upstreamUrl: https://mcp.crm-vendor.example/mcp
    upstreamAuthHeader: "Authorization: Bearer ${CRM_TOKEN}"
    policy: ["./policies/crm.yaml"]
    expectVersion: "0.3.0"        # fail closed if the policy drifts
  - name: warehouse                # -> POST /mcp/warehouse
    transport: stdio
    command: npx
    args: ["-y", "@modelcontextprotocol/server-postgres", "$DW_URL"]
    policy: ["./policies/warehouse.yaml"]
```

Run it audit-only for a week; the shared tape is the first place the **union** of an agent's behavior across systems is visible at once. Then draft, review, and flip routes to enforcement one at a time. A representative route policy reads broadly, writes narrowly, redacts what the model should never see, and refuses the risky sequence:

```
# crm.yaml (excerpt) — read broad, write narrow, PII redacted
capabilities:
  - target: "tool:query_*"
    actions: [call]
    directives:
      - type: redactFields
        fields: ["contact.email", "contact.phone"]
    conditions:
      - { type: maxCalls, count: 200, windowSeconds: 3600 }
  - target: tool:update_record
    actions: [call]
```

```
    conditions:
      - type: allowedValues
        argument: recordType
        values: ["case", "task"]   # never account / opportunity
      - { type: maxCalls, count: 25, windowSeconds: 3600 }
      - type: sequenceBlock
        afterTools: [query_sensitive]
  # delete_record, bulk_*, export_* : denied by absence
```

Layer the IdP on top (`--jwks-uri`, `--jwt-issuer`, `--jwt-audience`) so every request to the gateway must carry a bearer token your IdP signed.

What you get. SOC 2 evidence host logs and vendor logs cannot produce: one verifiable tape, each record stamped with the route, the policy version, and the policy's SHA-256 digest — **which** policy allowed **which** call, provably unedited. Policies live in git and are reviewed like code.

Graduate when an agent enters a customer-facing request path, or a second gateway appears in another team.

4.3 Large enterprise (500+ engineers, agents in the request path)

Situation. Many teams, many agents, some serving customers; agent platforms that fan a single task across CRM, ITSM, messaging, and a warehouse at once; availability requirements that rule out single-instance anything; a security organization that thinks in fleets.

Deployment. Three additions to the mid-size shape:

- **Multi-instance enforcement state.** Run gateways HA behind a load balancer and point every instance at one Redis (`--redis-addr`) so call budgets and session-sequence history are shared rather than per-instance. The kill switch rides the same backend — and deliberately **fails closed during a Redis outage**, denying rather than forgetting a kill order.
- **The orchestrator pattern.** Where one agent touches many systems in a single task, the gateway's shared tape is the only record in the organization that shows the cross-system union — “did any agent read PII from system A and write it to system B?” is unanswerable from any single host's or server's logs. Sequence rules then act on that union across routes.
- **The fleet.** At tens of gateways the operating questions change: who edits which policy, how a new version reaches every proxy, how tapes roll up into the SIEM, how you revoke every active agent session at once. That is eunox's **commercial fleet control plane** — central policy distribution with signed pulls, fleet inventory, organization-wide kill, cross-host audit aggregation, and compliance reporting.

> **The open-source line is explicit.** Everything that runs on your infrastructure — the proxy, every enforcement feature, every transport, the audit chain and its verifier — is Apache-2.0, free, at any scale, permanently. The commercial product is only the control plane that turns many proxies into one governed fleet.

4.4 Regulated institutions (banking, insurance, healthcare, government)

Regulation is an overlay, not a size — a forty-person broker-dealer inherits this profile at seat one. What defines the deployment is the constraint envelope: examination-grade audit, the strictest fail-closed posture, and a hard preference for nothing leaving the perimeter.

Deployment. Entirely inside the VPC or on-premises, often air-gapped — viable because the proxy is one static binary that calls nothing home. The destination posture is aggressively deny-by-default: reads

allowlisted and bounded, every mutating tool denied by absence (mutations go through a human ticket), sensitive fields redacted before they reach the model, and grants that expire rather than live forever:

```
# core-banking-readonly.yaml — examiner-grade, deny-by-default
capabilities:
  - target: "tool:get_*"
    actions: [call]
    directives:
      - type: redactFields
        fields: ["account.pan", "customer.ssn", "customer.dob"]
    conditions:
      - type: ipRange
        cidrs: ["10.20.0.0/16"]      # only the agent subnet
      - type: timeWindow
        notBefore: "2026-01-01T00:00:00Z"
        notAfter:  "2026-12-31T23:59:59Z"   # re-attested yearly
      - { type: maxCalls, count: 500, windowSeconds: 3600 }
  # every write / transfer / adjust tool: denied by absence
```

The audit tape is the deliverable. The HMAC-chained log plus `eunox audit-verify` gives an examiner what application logs cannot: a machine-checkable proof that the record of every agent decision was not edited after the fact. Fail-closed-on-ambiguity and the kill switch map directly onto the incident-response controls model-risk reviewers expect. For this buyer the core properties are not features; they are prerequisites.

At the fleet line, a hosted control plane is typically a non-starter; the fleet surface is available as a **self-hosted** control-plane edition run on the institution's own infrastructure.

4.5 The producer pattern: fronting your own MCP server

Everything above is the consumer view — governing your own agents. The mirror case applies to any company exposing an MCP server for **other people's** agents: put eunox in front of your own server as a reference monitor. Bound what any external agent may do per token, and hold a signed decision record independent of your application logs. The gateway shape is identical; JWT validation means a token you issue can only narrow the route's policy, and standard OAuth protected-resource metadata (RFC 9728) lets the proxy take its place in your existing authorization topology. Your customers can run the same binary on their side of the connection — one policy grammar governing both directions of trust.

5. Choosing your deployment

	Small team	Mid-size	Enterprise	Regulated
Shape	stdio wiretap; single guard before a remote server	Team gateway, one tape, per-route policies	HA gateways + Redis; fleet control plane	On-prem / air-gapped gateway
Policy posture	Reference policies; deny-by-absence	Drafted from the tape; reviewed in git	Least privilege + cross-route sequence rules	Aggressive deny-by-default; redaction; expiring grants
Identity	None needed	IdP JWT validation	IdP + token-claim narrowing	Per-environment IdP pinning
State	In-memory	In-memory	Redis: shared budgets, kill switch	Per posture; Redis if HA
Evidence	Local signed tape	Tape + policy version/digest per record (SOC 2)	SIEM roll-up across the fleet	audit-verify as examiner artifact
Beyond OSS	Nothing	Nothing	Commercial control plane	Self-hosted control-plane edition

Two rules of thumb carry most of the table. **The wiretap is the demo; the gateway is the deployment** — graduate at the second server, which is almost immediately. And **multiple proxy instances need Redis** — a load-balanced gateway without shared state keeps quotas and session history per-instance.

6. Common objections, answered

“Our OAuth setup already gates MCP access.”

It gates admission, at the granularity of scopes, once per token. It does not bound arguments, budgets, or sequences inside the granted scope; it does not exist at all on the local stdio path; and it produces no decision record. Keep it — eunox validates the very tokens it issues and enforces beneath them.

“Our MCP host has permission prompts.”

Per-host, per-machine, click-through, and blind to every other host's servers. An organization cannot build enforcement — or evidence — on each employee's client configuration. The gateway is host-independent and its tape is verifiable.

“We'll wait until agents are really in production.”

The wiretap makes waiting unnecessary: audit-only, zero blast radius, reversible in one line — and it converts “do we have an agent-governance problem?” from a debate into a tape you can read.

“What does a proxy cost us at runtime?”

One process, one hop. Enforcement is an in-memory decision against a compiled manifest; audit writes are non-blocking (a background worker signs and persists). Published latency baselines ship with the project's benchmarks documentation.

“What doesn't it do?”

eunox does not detect prompt injection or jailbreaks — it bounds their blast radius. It is not an authorization server, a sandbox, or a human-approval workflow engine, and it does not inspect model inputs or outputs. It enforces what an agent may **do** at the MCP boundary and proves what it did. A published threat model states precisely what is in and out of scope — your security team can evaluate it before any commitment.

7. Getting started

- **Today:** put the wiretap under one agent — `eunox proxy --audit -- <server>` — and read the tape.
- **This week:** run `eunox stats` and `eunox suggest`; review the drafted manifest; enforce it for that one server.
- **This month:** stand up a gateway for one team — audit-only first, then flip routes to enforcement as policies are reviewed.
- **This quarter:** wire the IdP, set production bounds, add Redis if you run more than one instance — and decide, with the tape in hand, what the fleet needs.

> **eunox is open source.** The proxy, every condition type, every transport, the audit chain and verifier, and a library of reference policies for popular MCP servers are Apache-2.0. Source, documentation, threat model, and benchmarks: **github.com/eunolabs/eunox**